Tarea Individual DG_S4

Formulación y Búsqueda Ciega en Espacios de Estados

Justin Guevara

No Institute Given

Enlace al repositorio de GitHub: https://github.com/JUSTAILS/IA---JUSTIN-GUEVARA

Objetivos

General

- Formular un problema mediante un espacio de estados e implementar algoritmos de búsqueda ciega para resolverlo, comparando las estrategias BFS y DFS mediante métricas de exploración, tiempo de ejecución y longitud de la solución.

Específicos

- Definir los componentes formales del problema: estados, estado inicial, estado objetivo, acciones, función de transición y restricciones de validez.
- Implementar los algoritmos BFS y DFS en Python para explorar el espacio de estados y encontrar una ruta hacia el objetivo.
- Medir el comportamiento de ambos algoritmos mediante el número de nodos expandidos, los estados descubiertos, el tiempo de ejecución, la longitud de la solución y el factor de ramificación efectivo.
- Comparar los resultados experimentales para analizar las características y diferencias de ambas estrategias de búsqueda ciega.

Introducción

La inteligencia artificial comprende métodos que permiten representar problemas y encontrar soluciones mediante procedimientos computacionales. Entre sus fundamentos se encuentra la formulación de problemas como espacios de estados, en los cuales una configuración inicial puede transformarse en otras configuraciones mediante acciones definidas, hasta alcanzar un estado que satisfaga las condiciones del objetivo.

La búsqueda ciega, también denominada búsqueda no informada, constituye una familia de estrategias que exploran el espacio de estados sin emplear información heurística para estimar la proximidad al objetivo. Dentro de esta categoría, la

búsqueda en anchura (Breadth-First Search / BFS) y la búsqueda en profundidad (Depth-First Search / DFS) representan dos métodos relevantes, diferenciados principalmente por el orden en que procesan los estados pendientes de exploración.

Para estudiar estos conceptos, se utiliza el problema de los misioneros y caníbales, que consiste en trasladar tres misioneros y tres caníbales entre las dos orillas de un río mediante una embarcación con capacidad para dos personas. La solución debe respetar las restricciones que impiden que los caníbales superen en número a los misioneros en una orilla donde estos últimos estén presentes.

Este problema permite establecer formalmente el espacio de estados, definir las acciones posibles y descartar las transiciones que producen configuraciones no válidas. A partir de esta formulación, se implementan BFS y DFS para encontrar rutas hacia el estado objetivo y comparar experimentalmente su comportamiento.

Formulación del problema

Descripción del problema

- El problema de los misioneros y caníbales consiste en trasladar tres misioneros y tres caníbales desde la orilla izquierda hasta la orilla derecha de un río. Para realizar los cruces se dispone de una embarcación con capacidad para una o dos personas.

La solución debe satisfacer las siguientes condiciones:

1. La embarcación solo puede transportar uno o dos pasajeros por cruce.
2. La embarcación debe encontrarse en la orilla desde la cual se inicia cada movimiento.
3. En ninguna orilla donde haya misioneros puede haber más caníbales que misioneros.
4. La cantidad total de misioneros y caníbales debe conservarse durante todo el proceso.
5. El objetivo consiste en trasladar a las seis personas a la orilla derecha.

Estas condiciones permiten establecer un espacio de estados finito en el que los algoritmos de búsqueda pueden explorar las transiciones permitidas hasta encontrar una solución.

Representación de los estados

- Cada estado se representa mediante una tupla de tres componentes:

$$S = (M, C, B)$$

Donde:

(M): número de misioneros en la orilla izquierda. (C): número de caníbales en la orilla izquierda. (B): posición de la embarcación, definida como:

B=1, si el barco se encuentra en la orilla izquierda, y B=0, si el barco se encuentra en la orilla derecha.

Como existen tres misioneros y tres caníbales, las cantidades presentes en la orilla derecha se calculan mediante:

$$M_D = 3 - M$$
$$C_D = 3 - C$$

Esta representación permite describir la distribución de las personas y la posición de la embarcación sin almacenar por separado la información de ambas orillas.

Estado inicial y estado objetivo

- El estado inicial representa la situación anterior al primer cruce, cuando todas las personas y la embarcación se encuentran en la orilla izquierda:

$$S_0 = (3, 3, 1)$$

El estado objetivo se alcanza cuando todos los misioneros y caníbales están en la orilla derecha y la embarcación también se encuentra en ella:

$$S_G = (0, 0, 0)$$

Por tanto, el algoritmo debe encontrar una secuencia de transiciones válidas que conecte (S_0) con (S_G).

Acciones permitidas

- La embarcación tiene una capacidad máxima de dos pasajeros, por lo que en cada cruce debe transportar al menos una persona y como máximo dos. Los pasajeros pueden ser misioneros, caníbales o una combinación de ambos.

El conjunto de acciones posibles se representa mediante pares ordenados (m, c), donde m indica la cantidad de misioneros y c la cantidad de caníbales que participan en el cruce. Por tanto, el conjunto de acciones es:

$$A = (1, 0), (2, 0), (0, 1), (0, 2), (1, 1)$$

Cada elemento representa una combinación de pasajeros: un misionero, dos misioneros, un caníbal, dos caníbales o un misionero junto con un caníbal, respectivamente.

La dirección del desplazamiento depende de la posición de la embarcación, representada mediante la variable B del estado. Cuando $B = 1$, la embarcación se encuentra en la orilla izquierda y los pasajeros se trasladan hacia la derecha. Cuando $B = 0$, se encuentra en la orilla derecha y el desplazamiento se realiza en sentido contrario.

Sin embargo, no todas las acciones pueden ejecutarse desde cualquier estado. Para que un movimiento sea válido, debe existir en la orilla de partida una cantidad suficiente de pasajeros para realizarlo. Además, el estado resultante debe cumplir las restricciones de seguridad establecidas para el problema.

Función de transición

- La función de transición determina el estado resultante después de ejecutar una acción sobre el estado actual. Su comportamiento depende de la posición de la embarcación, puesto que esta establece la orilla desde la cual se realiza el desplazamiento.

Sea el estado actual $S = (M, C, B)$ y una acción $a = (m, c)$, donde m representa la cantidad de misioneros y c la cantidad de caníbales que participan en el cruce. La función de transición se define mediante dos expresiones, según la ubicación de la embarcación.

Cuando la embarcación se encuentra en la orilla izquierda ($B = 1$), los pasajeros se trasladan hacia la derecha y las cantidades de misioneros y caníbales de la orilla izquierda disminuyen:

$$T((M, C, 1), (m, c)) = (M - m, C - c, 0)$$

Por el contrario, cuando la embarcación se encuentra en la orilla derecha ($B = 0$), los pasajeros regresan a la izquierda y las cantidades correspondientes aumentan:

$$T((M, C, 0), (m, c)) = (M + m, C + c, 1)$$

En ambos casos, la variable B cambia de valor para representar el desplazamiento de la embarcación hacia la orilla opuesta.

La aplicación de esta función está condicionada por dos aspectos: primero, debe existir una cantidad suficiente de pasajeros en la orilla de partida para ejecutar la acción; segundo, el estado resultante debe satisfacer las restricciones de seguridad y los límites establecidos para las cantidades de misioneros y caníbales. De esta manera, únicamente se consideran transiciones que conducen a estados válidos dentro del espacio de búsqueda.

Restricciones de validez

- Para que un estado sea considerado válido, debe cumplir las condiciones que garantizan la seguridad de los misioneros en ambas orillas del río. La restricción principal establece que los caníbales no pueden superar en número a los misioneros en una orilla cuando estos últimos se encuentran presentes.

Considerando un estado $S = (M, C, B)$, donde M y C representan las cantidades de misioneros y caníbales en la orilla izquierda, las cantidades correspondientes a la orilla derecha se obtienen mediante:

$$M_D = 3 - M$$

$$C_D = 3 - C$$

A partir de esta representación, se establecen las siguientes condiciones:

1. Límites de población: las cantidades de misioneros y caníbales en la orilla izquierda deben encontrarse dentro del intervalo de cero a tres:

$$0 \leq M \leq 3$$

$$0 \leq C \leq 3$$

Además, la variable que representa la posición de la embarcación únicamente puede tomar los valores cero o uno:

$$B \in 0, 1$$

2. Seguridad en la orilla izquierda: si existe al menos un misionero en esta orilla, su cantidad debe ser igual o superior a la de los caníbales:

$$M = 0 \quad \text{o} \quad M \geq C$$

3. Seguridad en la orilla derecha: se debe aplicar el mismo criterio en la orilla opuesta:

$$M_D = 0 \quad \text{o} \quad M_D \geq C_D$$

Estas condiciones se verifican después de cada transición. Si alguna de ellas no se cumple, el estado resultante se considera inválido y no debe incorporarse al espacio de búsqueda.

De esta manera, las restricciones permiten descartar los estados que representan situaciones prohibidas y garantizan que los algoritmos de búsqueda exploren únicamente configuraciones válidas del problema.

Criterios de solución

- Una solución para el problema de los misioneros y caníbales consiste en encontrar una secuencia finita de estados que permita trasladar a todos los individuos desde la orilla izquierda hasta la orilla derecha, respetando las restricciones establecidas durante cada cruce.

La secuencia debe comenzar en el estado inicial $S_0 = (3, 3, 1)$ y finalizar en el estado objetivo $S_G = (0, 0, 0)$. Por tanto, una solución puede representarse de la siguiente manera:

$$S_0 \to S_1 \to S_2 \to \cdots \to S_k = S_G$$

Para que esta secuencia sea considerada válida, debe cumplir las siguientes condiciones:

El primer estado debe corresponder a la configuración inicial del problema. Cada estado sucesor debe obtenerse mediante la aplicación de una acción permitida sobre el estado anterior. Todos los estados de la secuencia deben satisfacer las restricciones de seguridad y los límites de población. El último estado debe coincidir con el estado objetivo, indicando que los tres misioneros, los tres caníbales y la embarcación se encuentran en la orilla derecha.

En este problema, cada cruce de la embarcación tiene el mismo costo, independientemente de la cantidad y combinación de pasajeros transportados. Por consiguiente, el costo total de una solución corresponde al número de transiciones realizadas, representado por k.

Este criterio permite establecer una diferencia entre los algoritmos de búsqueda implementados. La búsqueda en anchura (BFS), al explorar los estados por niveles, garantiza encontrar una solución con el menor número de cruces cuando todas las acciones tienen el mismo costo. Por otra parte, la búsqueda en profundidad (DFS) permite encontrar una ruta válida, pero no garantiza que esta sea la de menor longitud.

De esta manera, el objetivo de la implementación consiste en encontrar una secuencia de transiciones válidas que alcance el estado objetivo y analizar el comportamiento de ambos algoritmos durante el proceso de búsqueda.

Implementación en Python

Implementación de los algoritmos de búsqueda

- La formulación del problema como un espacio de estados constituye la base para desarrollar su solución computacional. En esta etapa se implementaron los elementos definidos anteriormente mediante el lenguaje de programación Python, estableciendo mecanismos para representar las configuraciones del problema, generar transiciones válidas y explorar el espacio de búsqueda.

La implementación contempla dos algoritmos de búsqueda ciega: búsqueda en anchura (BFS) y búsqueda en profundidad (DFS). Ambos utilizan la misma representación de estados, las mismas acciones permitidas y las mismas restricciones de seguridad, diferenciándose principalmente en la estrategia utilizada para explorar los estados disponibles.

Representación de los estados y acciones

- Cada estado del problema se representa mediante una tupla de tres elementos (M, C, B), donde M y C indican las cantidades de misioneros y caníbales que permanecen en la orilla izquierda, mientras que B identifica la posición de la embarcación. Esta representación permite almacenar cada configuración de manera compacta y facilita la identificación de estados repetidos durante la búsqueda.

El estado inicial se establece mediante la tupla $(3, 3, 1)$, mientras que el estado objetivo corresponde a $(0, 0, 0)$. Asimismo, las cantidades de individuos en la orilla derecha se calculan a partir de las cantidades presentes en la izquierda, considerando que la población total permanece constante.

Para determinar si una configuración puede formar parte del espacio de búsqueda, se implementó la función estado_valido(). Esta función verifica que las cantidades de misioneros y caníbales se encuentren dentro de los límites establecidos, que la posición de la embarcación sea válida y que se respeten las restricciones de seguridad en ambas orillas.

Por otra parte, la función obtener_sucesores() se encarga de generar los estados que pueden alcanzarse a partir de una configuración actual. Para ello, recorre el conjunto de acciones permitidas y comprueba que exista una cantidad suficiente de pasajeros en la orilla de partida. Posteriormente, calcula el nuevo estado según la dirección del desplazamiento y utiliza la función estado_valido() para determinar si puede incorporarse como sucesor.

De esta manera, la generación de sucesores evita incorporar configuraciones que incumplan las condiciones del problema. Ambos algoritmos utilizan este mismo procedimiento, garantizando que la comparación de sus estrategias de búsqueda se realice sobre un espacio de estados común.

Busqueda en anchura (BFS)

- La búsqueda en anchura (Breadth-First Search, BFS) se implementó como el primer método de exploración del espacio de estados. Este algoritmo utiliza una estructura de datos tipo cola para procesar los estados según el orden en que fueron descubiertos, siguiendo una estrategia de primero en entrar, primero en salir (FIFO).

Para su implementación en Python se utilizó la estructura deque, perteneciente al módulo collections, que permite insertar nuevos estados al final de la cola y extraerlos desde el inicio de manera eficiente.

El procedimiento comienza incorporando el estado inicial $(3, 3, 1)$ a la cola de exploración y al conjunto de estados visitados. Posteriormente, mientras existan estados pendientes, se extrae el primero de la cola y se comprueba si corresponde al estado objetivo. Si no se ha alcanzado la solución, se generan sus sucesores mediante la función obtener_sucesores().

Cada sucesor que todavía no ha sido visitado se incorpora al conjunto de estados visitados y se añade al final de la cola. Adicionalmente, se registra su estado predecesor mediante una estructura de tipo diccionario. Este registro permite conservar la relación entre los estados y reconstruir posteriormente la secuencia de transiciones que conduce a la solución.

Cuando se encuentra el estado objetivo, el algoritmo utiliza la información almacenada de los predecesores para reconstruir la ruta desde el estado final hasta el inicial. Finalmente, invierte la secuencia obtenida para presentarla en el orden en que deben realizarse los cruces.

El uso de una cola FIFO garantiza que los estados se exploren por niveles de profundidad. Por esta razón, dado que todas las transiciones tienen el mismo costo, BFS permite encontrar una solución con el menor número de cruces posible.

Si se agotan los estados pendientes sin alcanzar el objetivo, el algoritmo determina que no se encontró una ruta de solución dentro del espacio de búsqueda explorado.

Busqueda en profundidad (DFS)

La búsqueda en profundidad (Depth-First Search, DFS) se implementó como el segundo método de exploración del espacio de estados. A diferencia de BFS, este algoritmo utiliza una estructura de datos tipo pila, siguiendo una estrategia de último en entrar, primero en salir (LIFO).

En Python, la pila se implementó mediante una lista, utilizando la operación append() para incorporar nuevos estados y pop() para extraer el último estado agregado. De esta manera, el algoritmo prioriza la exploración de los sucesores más recientemente incorporados a la estructura.

El procedimiento comienza insertando el estado inicial $(3, 3, 1)$ en la pila y registrándolo como visitado. Mientras existan estados pendientes de exploración, se extrae el último elemento de la pila y se comprueba si corresponde al estado objetivo. Si todavía no se ha alcanzado la solución, se generan sus sucesores mediante la función obtener_sucesores().

Los sucesores que no han sido visitados se incorporan a la pila y se registran junto con su estado predecesor. Al igual que en BFS, esta información permite reconstruir la ruta encontrada cuando se alcanza el estado objetivo. Una vez obtenida, la secuencia se invierte para mostrar los estados desde la configuración inicial hasta la configuración final.

La estrategia LIFO permite que DFS explore una trayectoria en profundidad antes de continuar con otras alternativas pendientes. Esto puede facilitar el descubrimiento de una solución sin necesidad de explorar previamente todos los estados de niveles anteriores. Sin embargo, no garantiza encontrar una ruta con el menor número de transiciones, incluso cuando existe una solución más corta.

En esta implementación, el conjunto de estados visitados evita explorar repetidamente una misma configuración, mientras que el registro de predecesores permite recuperar la ruta correspondiente a la primera solución encontrada. De esta manera, DFS utiliza los mismos mecanismos de validación y generación de sucesores que BFS, pero modifica el orden en que se explora el espacio de búsqueda.

Metricas de evaluación

Para analizar el comportamiento de los algoritmos de búsqueda implementados, se establecieron métricas que permiten evaluar el esfuerzo de exploración, el tamaño de las rutas obtenidas y el tiempo requerido para encontrar una solución. Estos indicadores facilitan la comparación entre BFS y DFS, considerando que ambos utilizan el mismo espacio de estados y las mismas restricciones.

Las métricas consideradas son las siguientes:

1. Nodos expandidos

Representa la cantidad de estados que fueron extraídos de la estructura de exploración y cuyos sucesores fueron generados y evaluados. Este indicador permite estimar el trabajo realizado por el algoritmo durante la búsqueda. No se considera expandido un estado cuando se reconoce como objetivo antes de generar sus sucesores.

2. Estados descubiertos

Corresponde a la cantidad total de estados únicos incorporados a la estructura de exploración durante la ejecución. Se incluye el estado inicial y se evita contabilizar nuevamente configuraciones que ya fueron visitadas. Esta métrica permite observar cuántos estados diferentes fueron identificados por cada algoritmo.

3. Longitud de la solución

Indica la cantidad de transiciones necesarias para alcanzar el estado objetivo desde el estado inicial. Se obtiene restando uno al número total de estados que conforman la ruta encontrada:

$$L = |R| - 1$$

Donde R representa la secuencia de estados que constituye la solución. Debido a que cada transición corresponde a un cruce de la embarcación, esta métrica también representa el número de cruces realizados.

4. Tiempo de ejecución

Mide el tiempo que necesita cada algoritmo para completar la búsqueda y obtener una solución. Para su medición en Python se utiliza la función perf_counter() del módulo time, que permite registrar intervalos de tiempo con alta resolución.

El tiempo de ejecución se calcula mediante la diferencia entre los instantes registrados al finalizar y comenzar el algoritmo:

$$T_e = t_{\text{final}} - t_{\text{inicial}}$$

Este indicador permite comparar el tiempo requerido por cada estrategia de búsqueda bajo las mismas condiciones de ejecución.

5. Factor de ramificación efectivo

El factor de ramificación efectivo (b^*) es un indicador que permite representar el crecimiento de los nodos descubiertos en relación con la profundidad de la solución. Se obtiene mediante la siguiente expresión:

$$N = 1 + b^* + (b^*)^2 + \cdots + (b^*)^d$$

Donde N representa el número total de estados descubiertos, incluyendo el estado inicial, y d corresponde a la profundidad de la solución, equivalente a la cantidad de transiciones realizadas.

El valor de b^* se determina mediante una aproximación numérica que satisface la igualdad anterior. Este indicador permite describir el esfuerzo de exploración en función de la profundidad alcanzada; sin embargo, debe interpretarse considerando que se trata de una estimación y que el espacio de estados del problema es finito.

En conjunto, estas métricas permiten evaluar tanto la eficiencia de exploración como la calidad de las soluciones obtenidas. Su utilización proporciona una base cuantitativa para analizar las diferencias entre BFS y DFS durante la resolución del problema de los misioneros y caníbales.

Resultados experimentales

Una vez implementados los algoritmos de búsqueda en anchura (BFS) y búsqueda en profundidad (DFS), se realizaron las ejecuciones correspondientes utilizando la misma configuración del problema, el mismo estado inicial y las mismas condiciones de transición.

Los resultados obtenidos permiten analizar las rutas encontradas y comparar el comportamiento de ambos algoritmos mediante las métricas establecidas. Para ello, se registraron los estados recorridos, los nodos expandidos, los estados

descubiertos, la longitud de las soluciones, el tiempo de ejecución y el factor de ramificación efectivo.

Rutas encontradas por los algoritmos

Ambos algoritmos consiguieron encontrar una secuencia de transiciones válidas que conecta el estado inicial $(3, 3, 1)$ con el estado objetivo $(0, 0, 0)$. Sin embargo, las rutas obtenidas presentan diferencias en algunas de las configuraciones intermedias debido al orden de exploración utilizado por cada estrategia.

La siguiente tabla muestra los estados que conforman las soluciones encontradas. Cada fila representa una etapa del recorrido, donde el número de cruce indica la cantidad de transiciones realizadas desde el estado inicial.

Tabla de rutas encontradas

Cruce	BFS	DFS
0	(3,3,1)	(3,3,1)
1	(3,1,0)	(2,2,0)
2	(3,2,1)	(3,2,1)
3	(3,0,0)	(3,0,0)
4	(3,1,1)	(3,1,1)
5	(1,1,0)	(1,1,0)
6	(2,2,1)	(2,2,1)
7	(0,2,0)	(0,2,0)
8	(0,3,1)	(0,3,1)
9	(0,1,0)	(0,1,0)
10	(1,1,1)	(0,2,1)
11	(0,0,0)	(0,0,0)

Se observa que ambas rutas alcanzan el estado objetivo después de realizar 11 cruces. Aunque existen diferencias en algunas configuraciones intermedias, las dos secuencias cumplen las restricciones de seguridad y las condiciones establecidas para el desplazamiento de la embarcación.

En el caso de BFS, la solución encontrada corresponde a una ruta de longitud mínima, debido a que el algoritmo explora los estados por niveles y todas las transiciones tienen el mismo costo. Por su parte, DFS también encontró una solución de igual longitud en esta ejecución particular; sin embargo, este resultado no representa una garantía del algoritmo, ya que su estrategia de exploración no asegura encontrar rutas óptimas.

Comparación de métricas

Para evaluar el comportamiento de ambos algoritmos se recopilaron los valores obtenidos durante la ejecución del programa. La comparación considera tanto el

esfuerzo de exploración como la longitud de las rutas y el tiempo requerido para encontrar una solución.

Métrica	BFS	DFS
¿Existe ruta?	Sí	Sí
Nodos expandidos	14	11
Estados descubiertos	15	15
Longitud de la solución	11	11
Tiempo de ejecución (segundos)	0.000230800	0.000066200
Factor de ramificación efectivo	1.039700	1.039700

Los resultados muestran que ambos algoritmos consiguieron resolver el problema y descubrieron la misma cantidad de estados. No obstante, se identifican diferencias en el número de nodos expandidos: BFS expandió 14 estados, mientras que DFS expandió 11. Esto indica que, durante esta ejecución, DFS necesitó examinar una menor cantidad de estados antes de encontrar el objetivo.

Respecto a la longitud de las soluciones, ambos algoritmos obtuvieron rutas de 11 transiciones. Este resultado permite observar que DFS encontró, en esta ejecución, una solución con la misma longitud que la obtenida mediante BFS, aunque no posee una garantía general de optimalidad.

En cuanto al tiempo de ejecución, BFS registró aproximadamente 0.000231 segundos, mientras que DFS registró 0.0000662 segundos. Por tanto, en esta ejecución DFS presentó un menor tiempo de procesamiento. Sin embargo, debido a que los tiempos registrados son muy pequeños y corresponden a una ejecución individual, esta diferencia debe interpretarse como un resultado puntual y no como una conclusión definitiva sobre el rendimiento general de ambos algoritmos.

Finalmente, el factor de ramificación efectivo presentó el mismo valor para ambos métodos, equivalente a 1.039700. Esto se debe a que su cálculo utiliza la cantidad de estados descubiertos y la profundidad de la solución, valores que fueron iguales en las dos ejecuciones.

Conclusiones

- La formulación del problema de los misioneros y caníbales como un espacio de estados permitió representar de manera estructurada las diferentes configuraciones posibles y establecer las condiciones necesarias para alcanzar una solución. La definición del estado, las acciones permitidas, la función de transición y las restricciones de seguridad constituyeron los elementos fundamentales para desarrollar la implementación computacional.
- La implementación de los algoritmos de búsqueda en anchura (BFS) y búsqueda en profundidad (DFS) permitió comprobar cómo dos estrategias de búsqueda ciega pueden explorar un mismo espacio de estados siguiendo órdenes diferentes. Ambos algoritmos consiguieron encontrar una solución válida,

alcanzando el estado objetivo mediante 11 transiciones y descubriendo 15 estados únicos durante sus respectivas ejecuciones.

- La comparación de las métricas evidenció diferencias en el esfuerzo de exploración. BFS expandió 14 nodos, mientras que DFS expandió 11, y este último registró un menor tiempo de ejecución en el experimento realizado. Sin embargo, estos resultados corresponden a una ejecución particular y no permiten establecer que DFS sea generalmente más eficiente. Por otra parte, aunque ambos algoritmos encontraron soluciones con la misma longitud, BFS garantiza una solución de longitud mínima cuando todas las transiciones tienen el mismo costo, mientras que DFS no ofrece esta garantía.